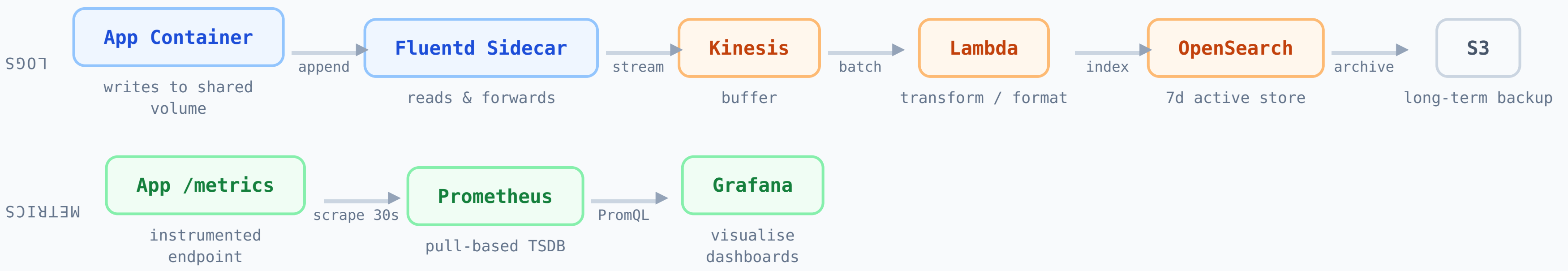
Logs vs Metrics — Why Two Pipelines



- variable-structure events · high cardinality · write-heavy
- fixed-schema time-series · low cardinality · aggregation-optimised

Why separate? Logs are append-only, schema-free, write-heavy streams. Metrics are fixed-schema, optimised for aggregation and range queries. Different compression, indexing, and retention strategies — one system cannot serve both efficiently.

Log Retention Tiers — Cost vs Query Speed

T1	**OpenSearch**	7 days retention · fast full-text query · active incident investigation
T2	**CloudWatch Logs**	30 days retention · medium cost · operational review & alerting
T3	**S3 / Glacier**	Years retention · cold storage · compliance, audit, legal hold

QUERY SPEED

ms ←————→ hours

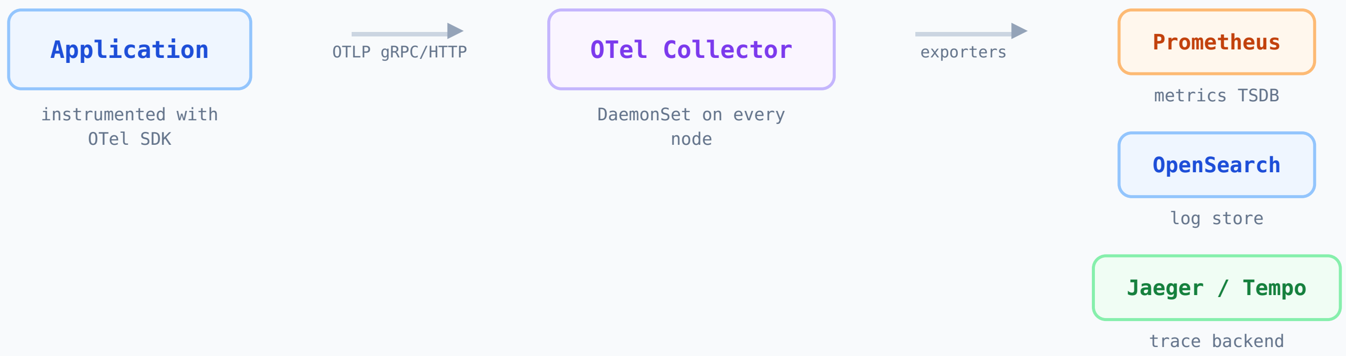
STORAGE COST / GB

$$$ ←————→ ¢

> Each tier trades **query speed ↔ storage cost**. OpenSearch keeps hot data addressable in milliseconds. S3 Glacier stores petabytes at cents per GB-month — retrieval takes minutes to hours. Match the tier to the access pattern, not the other way around.

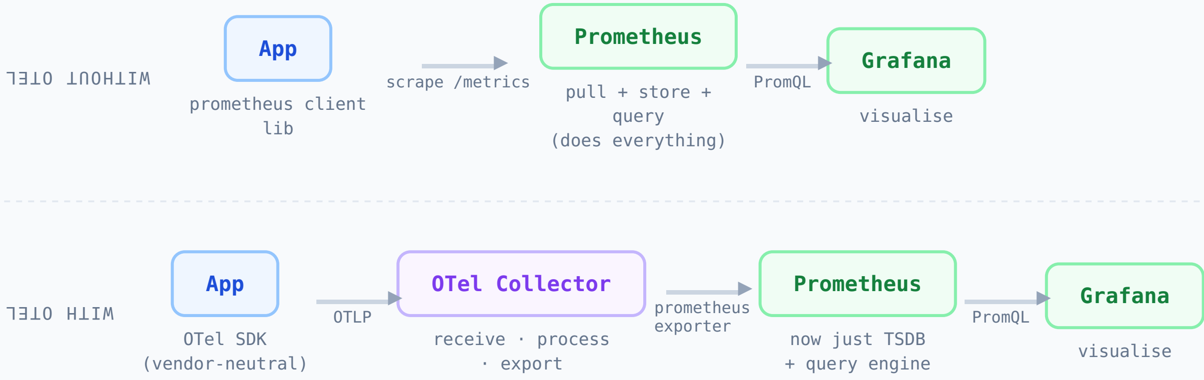
03 · COLLECTION LAYER

OpenTelemetry — One Pipeline, Three Signals



> OTel is **transport, not storage**. It receives logs, metrics, and traces via OTLP and exports to any backend. Swap Prometheus for Datadog by changing one exporter config — zero application code changes required.

Prometheus vs OTel — Different Layers of the Stack



Key shift: With OTel, Prometheus loses its scraping responsibility and becomes purely a storage + query engine. Instrumentation in your app is now decoupled from the backend — same code can ship to Datadog, VictoriaMetrics, or Thanos by changing one config line.

Datadog — Collapse the Entire Stack into One Agent

GAINS

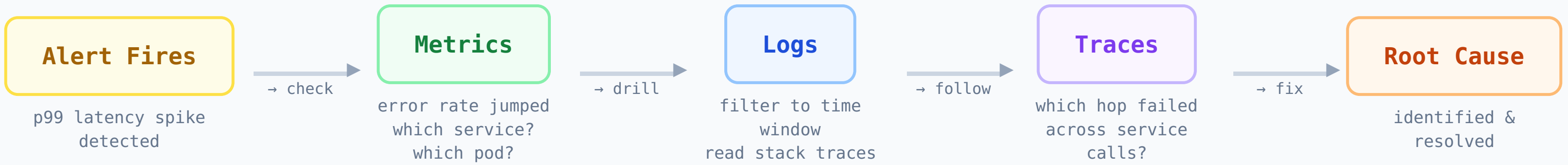
- Out-of-the-box signal correlation
- Auto-discovery on Kubernetes
- No backend infra to manage
- APM + profiling included

TRADEOFFS

- Vendor lock-in risk
- No control over storage tiers
- Cost scales with data volume
- Less flexible retention policy

When Datadog wins: When platform engineering headcount is limited. One agent, one backend, built-in correlation — you skip building and operating the Fluentd + Prometheus + OpenSearch + Jaeger stack entirely.

Three Pillars — How You Actually Debug a Production Issue



METRICS answer
What is degraded?
How bad is it?
Since when?

LOGS answer
What exactly happened?
What error was thrown?
What was the input?

TRACES answer
Where across services?
Which call was slow?
What was the chain?

> You cannot fully debug with only one or two pillars. Metrics show the symptom. Logs show what broke inside a service. Traces show where it broke across service boundaries. All three are required for distributed systems.

Tool Layer Mapping — OSS Stack vs OTel Stack vs Managed

LAYER	OSS STACK	WITH OTEL	MANAGED (DD / SPLUNK)
Collection	Fluentd + Prom scrape	OTel Collector	DD / Splunk Agent
Metrics DB	Prometheus	Prometheus	DD Metrics / Splunk
Log Store	OpenSearch	OpenSearch	Splunk / DD Logs
Trace Store	Jaeger / Tempo	Tempo	DD APM / Splunk APM
Visualise	Grafana	Grafana	DD / Splunk UI
All-in-one	—	—	Datadog or Splunk

> OTel does not replace tools — it inserts a **vendor-neutral collection layer** between your app and the backends. The backends (Prometheus, OpenSearch, Jaeger) remain unchanged. Datadog collapses all layers into one SaaS product at the cost of flexibility.